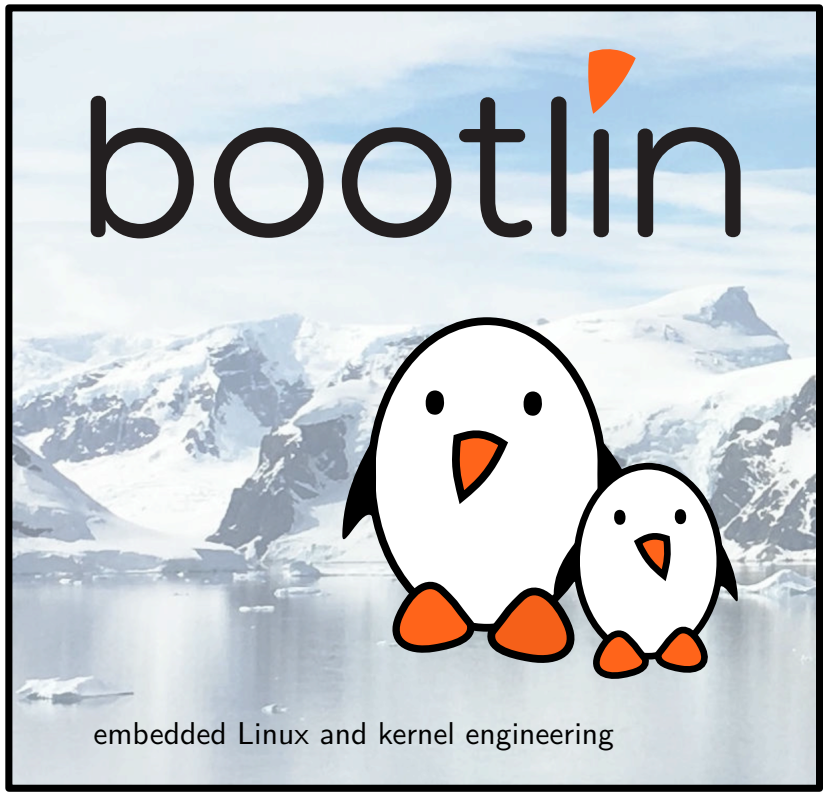
Describing hardware devices



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Discoverable hardware: USB and PCI



Discoverable hardware

- Some busses have built-in hardware discoverability mechanisms
- Most common busses: USB and PCI
- Hardware devices can be enumerated, and their characteristics retrieved with just a driver or the bus controller
- Useful Linux commands
  - `lsusb`, lists all USB devices detected
  - `lspci`, lists all PCI devices detected
  - A detected device does not mean it has a kernel driver associated to it!
- Association with kernel drivers done based on product ID/vendor ID, or some other characteristics of the device: device class, device sub-class, etc.



Describing non-discoverable hardware



Describing non-discoverable hardware

1. Directly in the **OS/bootloader code**

- Using compiled data structures, typically in C
- How it was done on most embedded platforms in Linux, U-Boot.
- Considered not maintainable/sustainable on ARM32, which motivated the move to another solution.



Describing non-discoverable hardware

2. Using **ACPI** tables

- On *x86* systems, but also on a subset of ARM64 platforms
- Tables provided by the firmware



Describing non-discoverable hardware

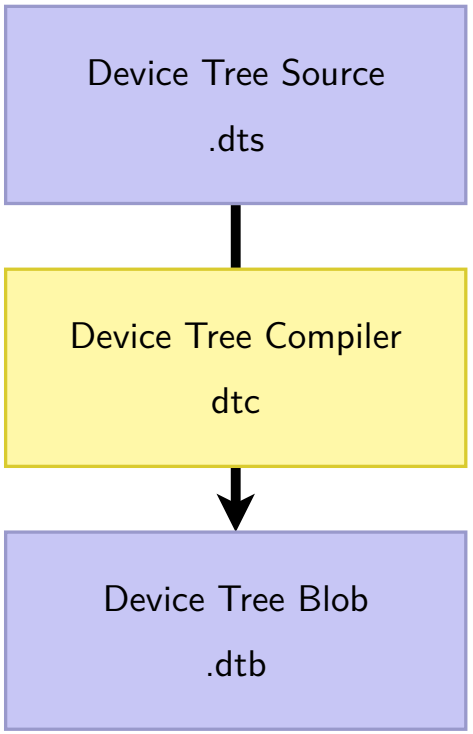
3. Using a **Device Tree**

- Originates from **OpenFirmware**, defined by Sun, used on SPARC and PowerPC
  - that's why many Linux/U-Boot functions related to DT have a `of_` prefix
- Now used by most embedded-oriented CPU architectures that run Linux: ARC, ARM64, RISC-V, ARM32, PowerPC, Xtensa, MIPS, etc.
- Writing/tweaking a DT is necessary when porting Linux to a new board, or when connecting additional peripherals



Device Tree: from source to blob

- A tree data structure describing the hardware is written by a developer in a **Device Tree Source** file, `.dts`
- Processed by the **Device Tree Compiler**, `dtc`
- Produces a more efficient representation: **Device Tree Blob**, `.dtb`
- Additional C preprocessor pass
- `.dtb` → accurately describes the hardware platform in an **OS-agnostic** way.
- `.dtb`≈ few dozens of kilobytes
- DTB also called **FDT**, *Flattened Device Tree*, once loaded into memory.
  - `fdt` command in U-Boot
  - `fdt_` APIs





dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```



dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```

```
$ dtc -I dtb -O dts foo.dtb
/dts-v1/;

/ {
        welcome = <0xbadcafe>;

        bootlin {
                webinar = "great";
                demo = <0x01 0x02 0x03>;
        };
};
```



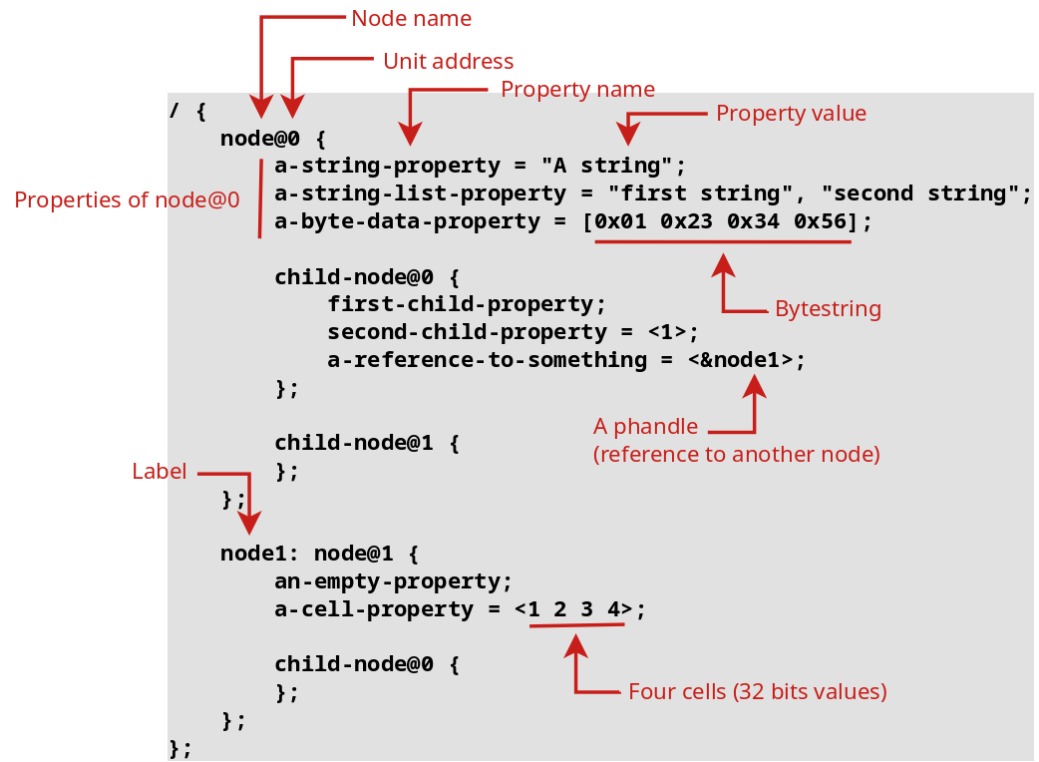
Where are Device Tree Sources located?

- Even though they are OS-agnostic, **no central and OS-neutral** place to host Device Tree sources and share them between projects
  - Often discussed, never done
- In practice, the Linux kernel sources can be considered as the **canonical location** for Device Tree Source files
  - `arch/<ARCH>/boot/dts/<vendor>/`
  - `arch/arm/boot/dts` (on ARM 32 architecture before Linux 6.5)
  - ≈ 4500 Device Tree Source files (`.dts` and `.dtsi`) in Linux as of 6.0.
- Duplicated/synced in various projects
  - U-Boot, Barebox, TF-A



Device Tree base syntax

- Tree of **nodes**
- Nodes with **properties**
- Node≈ a device or IP block
- Properties≈ device characteristics
- Notion of **cells** in property values
- Notion of **phandle** to point to other nodes
- `dtc` only does syntax checking, no semantic validation

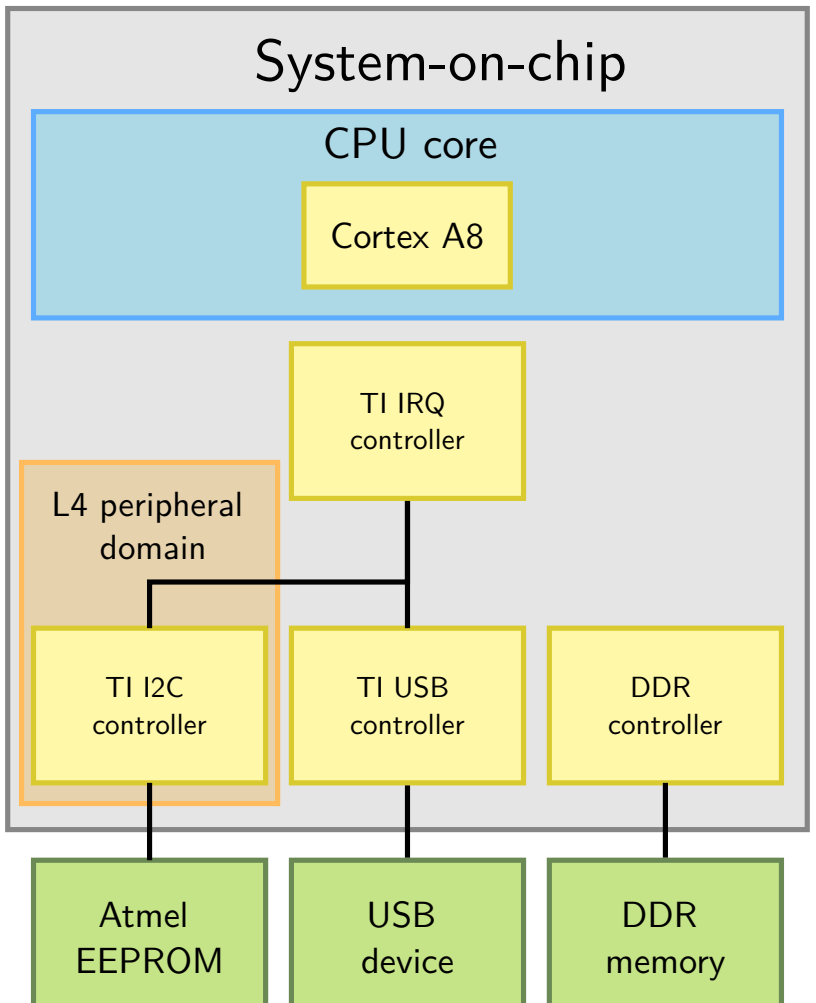




DT overall structure: simplified example

```
/ {
  #address-cells = <1>;
  #size-cells = <1>;
  model = "TI AM335x BeagleBone Black";
  compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";

  cpus { ... };
  memory@80000000 { ... };
  chosen { ... };
  ocp {
    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };
    l4_per: interconnect@44c00000 {
      i2c0: i2c@40012000 { ... };
    };
  };
};
```





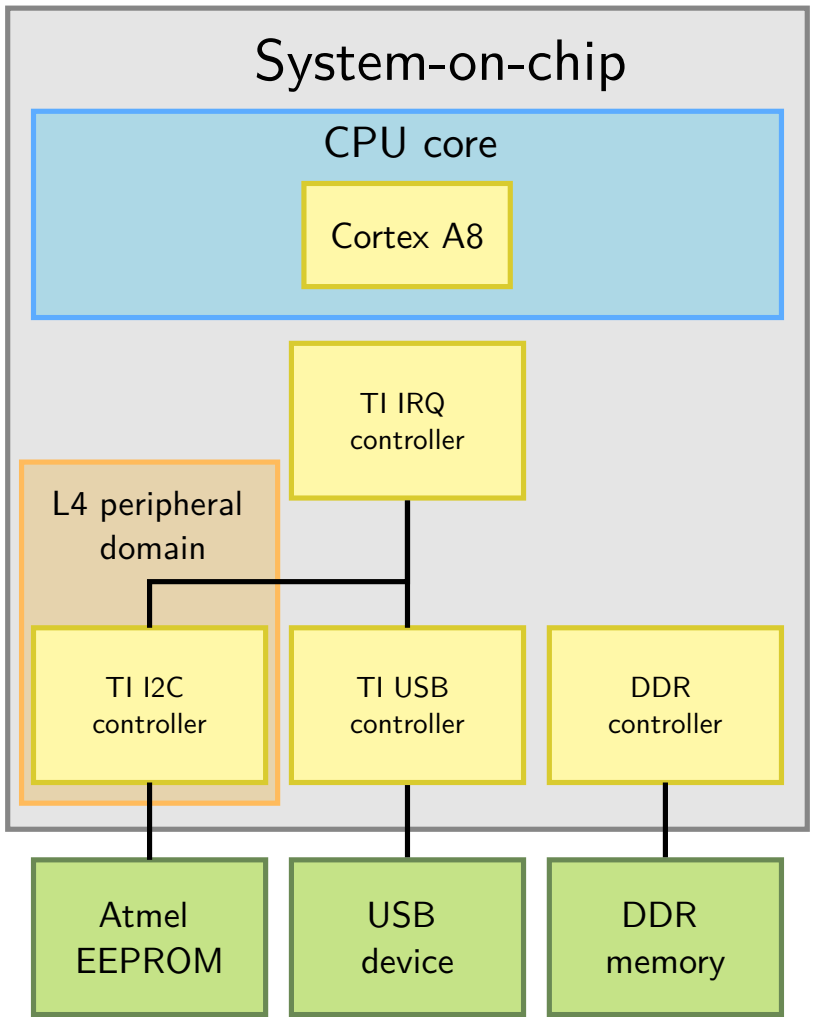
DT overall structure: simplified example

```
/ {
  cpus {
    #address-cells = <1>;
    #size-cells = <0>;
    cpu0: cpu@0 {
      compatible = "arm,cortex-a8";
      enable-method = "ti,am3352";
      device_type = "cpu";
      reg = <0>;
    };
  };

  memory@0x80000000 {
    device_type = "memory";
    reg = <0x80000000 0x10000000>; /* 256 MB */
  };

  chosen {
    bootargs = "";
    stdout-path = &uart0;
  };

  ocp { ... };
};
```





DT overall structure: simplified example

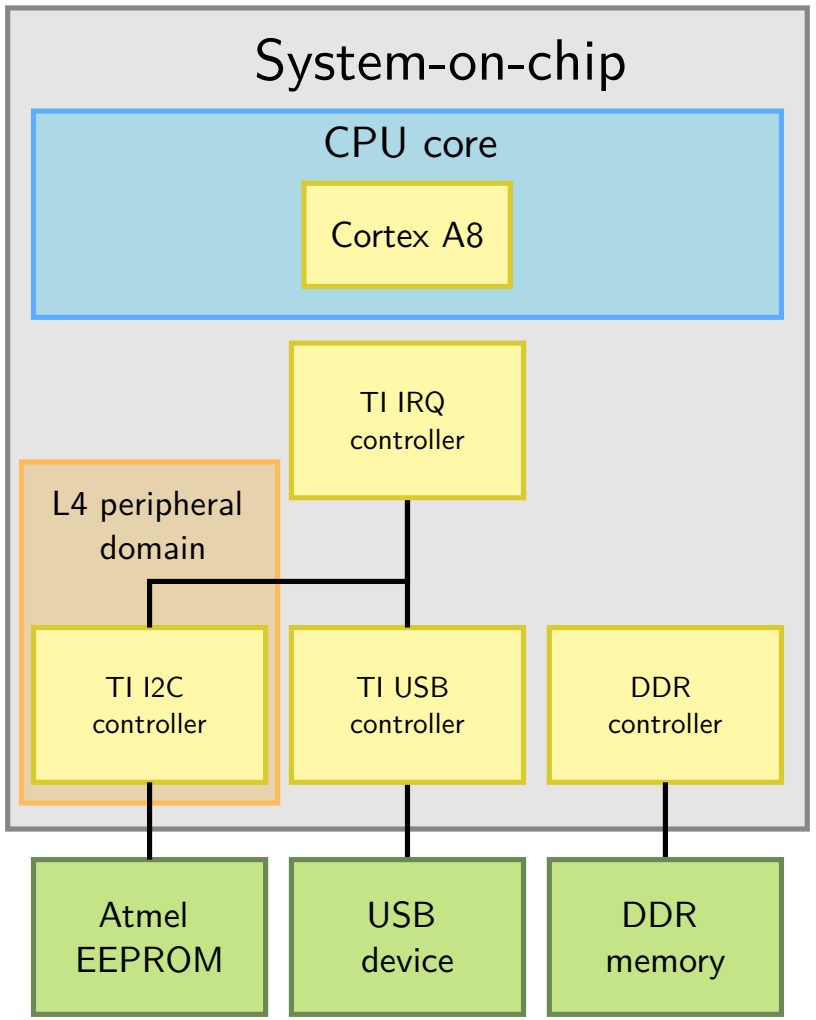
```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };
  ocp {

    intc: interrupt-controller@48200000 {
      compatible = "ti,am33xx-intc";
      interrupt-controller;
      #interrupt-cells = <1>;
      reg = <0x48200000 0x1000>;
    };

    usb0: usb@47401300 {
      compatible = "ti,musb-am33xx";
      reg = <0x1400 0x400>, <0x1000 0x200>;
      reg-names = "mc", "control";
      interrupts = <18>;
      dr_mode = "otg";
      dmas = <&cppi41dma 0 0>, <&cppi41dma 1 0>;
      status = "okay";
    };

    l4_per: interconnect@44c00000 {
      i2c0: i2c@40012000 { ... };
    };

  };
};
```



DT overall structure: simplified example

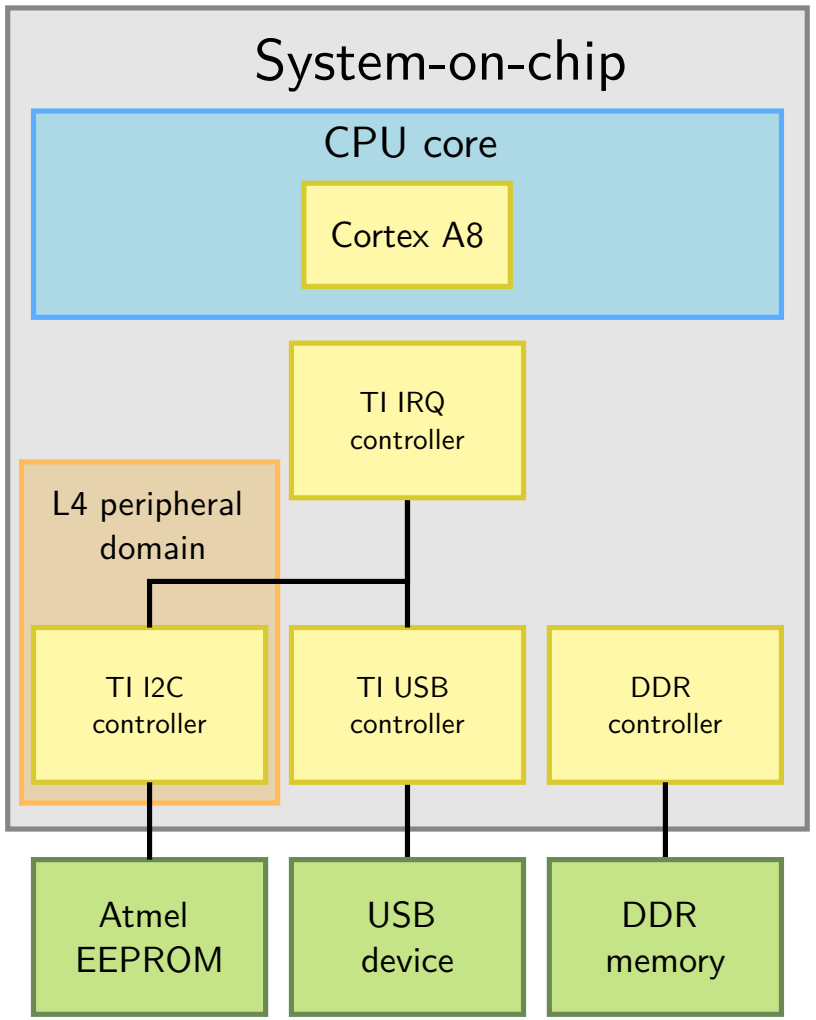
```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };

  ocp {
    compatible = "simple-pm-bus";
    clocks = <&l3_clkctrl AM3_L3_L3_MAIN_CLKCTRL 0>;
    clock-names = "fck";
    #address-cells = <1>;
    #size-cells = <1>;

    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };

    l4_per: interconnect@44c00000 {
      compatible = "ti,am33xx-l4-wkup", "simple-pm-bus";
      reg = <0x44c00000 0x800>, <0x44c00800 0x800>,
            <0x44c01000 0x400>, <0x44c01400 0x400>;
      reg-names = "ap", "la", "ia0", "ia1";
      #address-cells = <1>;
      #size-cells = <1>;

      i2c0: i2c@40012000 { ... };
    };
  };
};
```

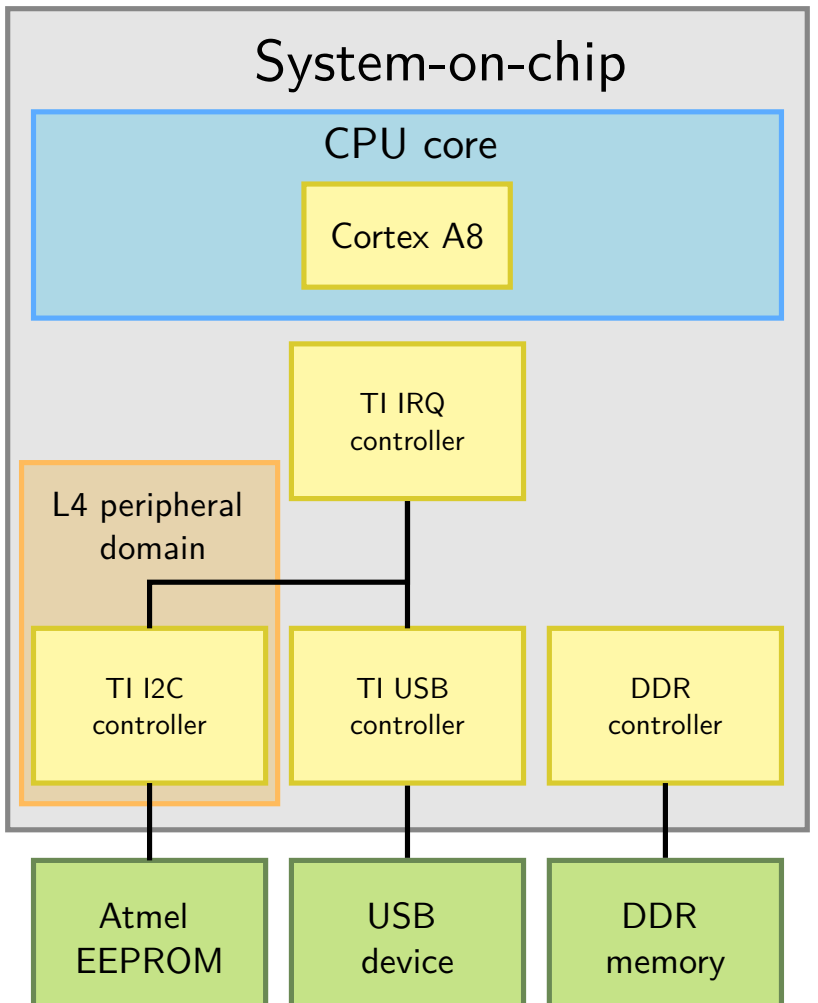


DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };
  ocp {
    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };
    l4_per: interconnect@44c00000 {

      i2c0: i2c@40012000 {
        compatible = "ti,omap4-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x0 0x1000>;
        interrupts = <70>;
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&i2c0_pins>;
        clock-frequency = <400000>;

        baseboard_eeprom: eeprom@50 {
          compatible = "atmel,24c256";
          reg = <0x50>;
        };
      };
    };
  };
};
```





Device Tree inheritance

- Device Tree files are not monolithic, they can be split in several files, including each other.
- `.dtsi` files are included files, while `.dts` files are *final* Device Trees
  - Only `.dts` files are accepted as input to `dtc`
- Typically, `.dtsi` will contain
  - definitions of SoC-level information
  - definitions common to several boards
- The `.dts` file contains the board-level information
- The inclusion works by **overlaying** the tree of the including file over the tree of the included file, according to the order of the `#include` directives.
- Allows an including file to **override** values specified by an included file.
- Uses the C pre-processor `#include` directive



Device Tree inheritance example

Definition of the AM33xx SoC family *(am33xx-l4.dtsi)*

```
&l4_wkup {
    target-module@b000 {
        i2c0: i2c@0 {
            compatible = "ti,omap4-i2c";
            reg = <0x0 0x1000>;
            interrupts = <70>;
            status = "disabled";
        };
    };
};
```

+

Definition of the Bone Black board *(am335x-boneblack.dts)*

```
#include "am33xx-l4.dtsi"

&i2c0 {
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pins>;
    status = "okay";

    baseboard_eeprom: eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;
    };
};
```

Note 1
The actual Device Trees for this platform are more complicated. This example is highly simplified.

=

Compiled DTB *(am335x-boneblack.dtb)*

```
&l4_wkup {
    target-module@b000 {
        i2c0: i2c@0 {
            compatible = "ti,omap4-i2c";
            reg = <0x0 0x1000>;
            interrupts = <70>;
            pinctrl-names = "default";
            pinctrl-0 = <&i2c0_pins>;
            status = "okay";

            baseboard_eeprom: eeprom@50 {
                compatible = "atmel,24c256";
                reg = <0x50>;
            };
        };
    };
};
```

Note 2
The real DTB is in binary format. Here we show the text equivalent of the DTB contents.



Inheritance and labels

Doing:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  ocp {
    serial@0 {
      status = "okay";
    };
  };
};
```



Inheritance and labels

Doing:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  ocp {
    serial@0 {
      status = "okay";
    };
  };
};
```

Is exactly equivalent to:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

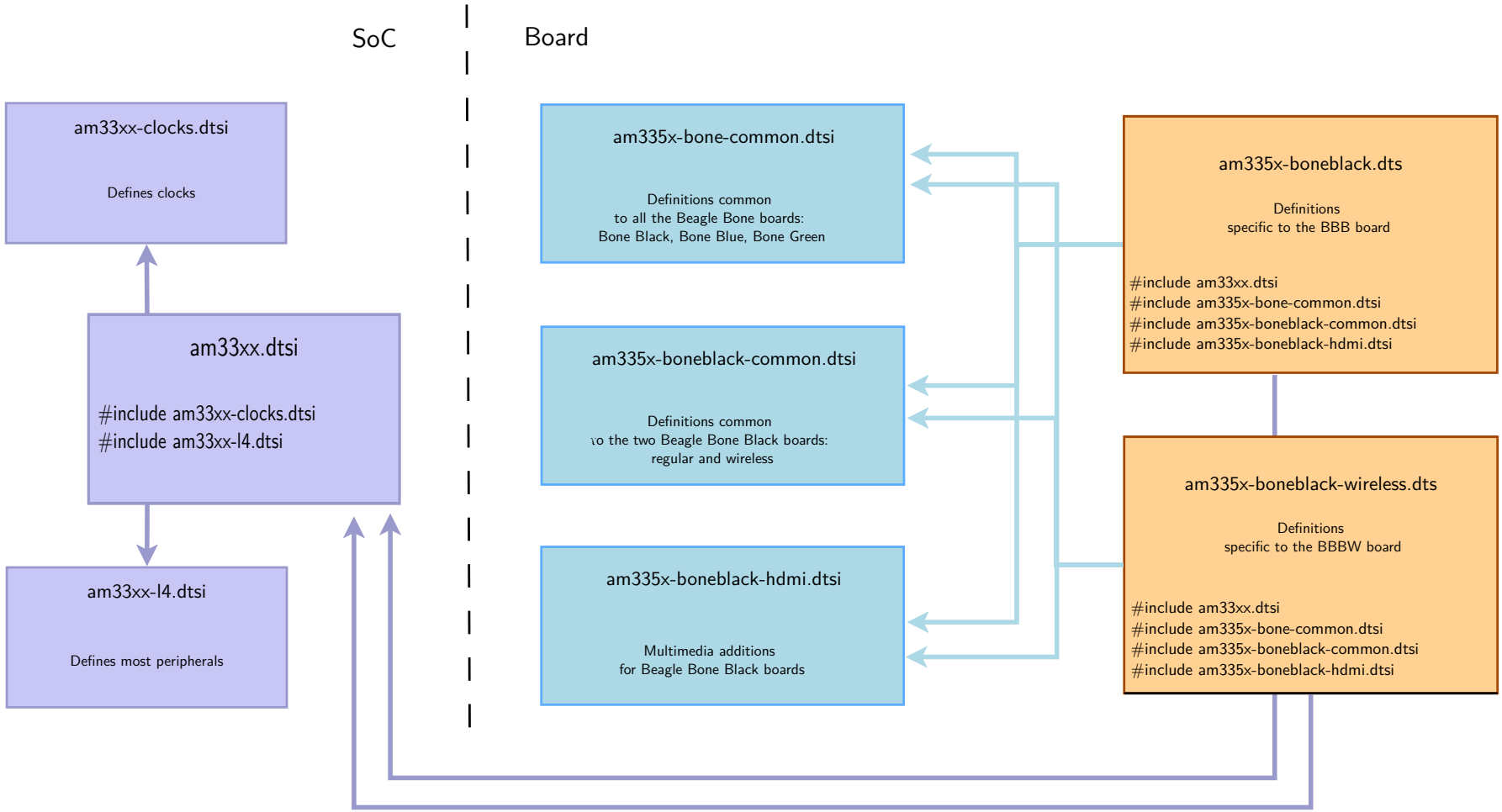
board.dts

```
#include "soc.dtsi"

&uart0 {
  status = "okay";
};
```

→ this solution is preferred

DT inheritance in Bone Black support





Device Tree design principles

- **Describe hardware** (how the hardware is), not configuration (how I choose to use the hardware)
- **OS-agnostic**
  - For a given piece of HW, Device Tree should be the same for U-Boot, FreeBSD or Linux
  - There should be no need to change the Device Tree when updating the OS
- Describe **integration of hardware components**, not the internals of hardware components
  - The details of how a specific device/IP block is working is handled by code in device drivers
  - The Device Tree describes how the device/IP block is connected/integrated with the rest of the system: IRQ lines, DMA channels, clocks, reset lines, etc.
- Like all beautiful design principles, these principles are sometimes violated.



The properties

Device tree properties can:

- Be generic and apply to most nodes
  - Their meaning is usually described in one place: the core DT schema available at https://github.com/devicetree-org/dt-schema.
  - `compatible`, `reg`, `#address-cells`, etc
- Cover common consumer-provider relationships
  - Their meaning is either described in the dt-schema GitHub repository or under Documentation/devicetree/bindings.
  - `clocks`, `interrupts`, `regulators`, etc
- Subsystem specific
  - All devices of a certain class may use them, often starting with the class name
  - `spi-cpha`, `i2c-scl-internal-delay-ns`, `nand-ecc-engine`, `mac-address`, etc
- Vendor/device specific
  - To describe uncommon or very specific properties
  - Always described in the device's binding file and prefixed with `<vendor>,`
  - `ti,hwmods`, `xlnx,num-channels`, `nxp,tx-output-mode`, etc
- Some of them are deprecated, watch out the bindings!



The `compatible` property

- Is a list of strings
  - From the most specific to the least specific
- Describes the specific **binding** to which the node complies.
- It uniquely identifies the **programming model** of the device.
- Practically speaking, it is used by the operating system to find the **appropriate driver** for this device.
- When describing real hardware, the typical form is `vendor,model`
- Examples:
  - `compatible = "arm,armv7-timer";`
  - `compatible = "st,stm32mp1-dwmac", "snps,dwmac-4.20a";`
  - `compatible = "regulator-fixed";`
  - `compatible = "gpio-keys";`
- Special value: `simple-bus` → bus where all sub-nodes are memory-mapped devices



`compatible` property and Linux kernel drivers

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - `struct of_device_id[]`
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.

```
/ {
    timer {                                  → Platform device
        compatible = "...";
    };
    soc {
        compatible = "simple-bus";
        uart@1000 {                          → Platform device
            compatible = "...";
        };
        i2c@2000 {                           → Platform device
            compatible = "...";
            eeprom@65 {                      → I2C device
                compatible = "...";
            };
        };
    };
};
```



Matching with drivers in Linux: platform driver

drivers/i2c/busses/i2c-omap.c

```
static const struct of_device_id omap_i2c_of_match[] = {
        {
                .compatible = "ti,omap4-i2c",
                .data = &omap4_pdata,
        },
        {
                .compatible = "ti,omap3-i2c",
                .data = &omap3_pdata,
        },
        [...]
        { },
};
MODULE_DEVICE_TABLE(of, omap_i2c_of_match);

[...]

static struct platform_driver omap_i2c_driver = {
        .probe          = omap_i2c_probe,
        .remove         = omap_i2c_remove,
        .driver         = {
                .name   = "omap_i2c",
                .pm     = &omap_i2c_pm_ops,
                .of_match_table = of_match_ptr(omap_i2c_of_match),
        },
};
```



Matching with drivers in Linux: I2C driver

sound/soc/codecs/cs42l51.c

```
const struct of_device_id cs42l51_of_match[] = {
        { .compatible = "cirrus,cs42l51", },
        { }
}; MODULE_DEVICE_TABLE(of, cs42l51_of_match);
```

sound/soc/codecs/cs42l51-i2c.c

```
static struct i2c_driver cs42l51_i2c_driver = {
        .driver = {
                .name = "cs42l51",
                .of_match_table = cs42l51_of_match,
                .pm = &cs42l51_pm_ops,
        },
        .probe = cs42l51_i2c_probe,
        .remove = cs42l51_i2c_remove,
        .id_table = cs42l51_i2c_id,
};
```



`reg` property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



`reg` property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.

```
&i2c1 {
   hdmi-transmitter@39 {
      reg = <0x39>;
   };
   cs42l51: cs42l51@4a {
      reg = <0x4a>;
   };
};
```



`reg` property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number

```
&qspi {
        flash0: mx66l51235l@0 {
                reg = <0>;
        };
        flash1: mx66l51235l@1 {
                reg = <1>;
        };
};
```



`reg` property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number
- The unit address must be the address of the first `reg` entry.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



`cells` property

- Property numbers shall fit into 32-bit containers called `cells`
- The compiler does not maintain information about the number of entries, the OS just receives 4 independent `cells`
  - Example with a `reg` property using 2 entries of 2 cells:

```
reg = <0x50027000 0x4>, <0x500273f0 0x10>;
```

  - The OS cannot make the difference with:

```
reg = <0x50027000>, <0x4>, <0x500273f0 0x10>;
reg = <0x50027000 0x4 0x500273f0>, <0x10>;
reg = <0x50027000>, <0x4 0x500273f0 0x10>;
reg = <0x50027000 0x4 0x500273f0 0x10>;
```



`cells` property

- Property numbers shall fit into 32-bit containers called `cells`
- The compiler does not maintain information about the number of entries, the OS just receives 4 independent `cells`
- Need for other properties to declare the right formatting:
  - `#address-cells`: Indicates the number of cells used to carry the address
  - `#size-cells`: Indicates the number of cells used to carry the size of the range
- The parent-node declares the children `reg` property formatting
  - Platform devices need memory ranges

```
module@a0000 {
    #address-cells = <1>;
    #size-cells = <1>;

    serial@1000 {
        reg = <0x1000 0x10>, <0x2000 0x10>;
    };
};
```



`cells` property

- Property numbers shall fit into 32-bit containers called `cells`
- The compiler does not maintain information about the number of entries, the OS just receives 4 independent `cells`
- Need for other properties to declare the right formatting:
  - `#address-cells`: Indicates the number of cells used to carry the address
  - `#size-cells`: Indicates the number of cells used to carry the size of the range
- The parent-node declares the children `reg` property formatting
  - Platform devices need memory ranges
  - SPI devices need chip-selects

```
spi@300000 {
    #address-cells = <1>;
    #size-cells = <0>;

    flash@1 {
        reg = <1>;
    };
};
```



Status property

- The `status` property indicates if the device is really in use or not
  - `okay` or `ok` → the device is really in use
  - any other value, by convention `disabled` → the device is not in use
- In Linux, controls if a device is instantiated
- In `.dtsi` files describing SoCs: all devices that interface to the outside world have: `status = "disabled";`
- Enabled on a per-device basis in the board `.dts`



Resources: interrupts, clocks, DMA, reset lines, ...

- Common pattern for resources shared by multiple hardware blocks
  - Interrupt lines
  - Clock controllers
  - DMA controllers
  - Reset controllers
  - ...
- A Device Tree node describing the *controller* as a device
- References from other nodes that use resources provided by this *controller*

```
intc: interrupt-controller@a0021000 {
   compatible = "arm,cortex-a7-gic";
   #interrupt-cells = <3>;
   interrupt-controller;
   reg = <0xa0021000 0x1000>, <0xa0022000 0x2000>;
};

rcc: rcc@50000000 {
   compatible = "st,stm32mp1-rcc", "syscon";
   reg = <0x50000000 0x1000>;
   #clock-cells = <1>;
   #reset-cells = <1>;
};

dmamux1: dma-router@48002000 {
   compatible = "st,stm32h7-dmamux";
   reg = <0x48002000 0x1c>;
   #dma-cells = <3>;
   clocks = <&rcc DMAMUX>;
   resets = <&rcc DMAMUX_R>;
};

spi3: spi@4000c000 {
   interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>;
   clocks = <&rcc SPI3_K>;
   resets = <&rcc SPI3_R>;
   dmas = <&dmamux1 61 0x400 0x05>,
          <&dmamux1 62 0x400 0x05>;
};
```



Generic suffixes

- `xxx-gpios`
  - When drivers need access to GPIOs
  - May be subsystem-specific or vendor-specific
  - Examples: `enable-gpios`, `cts-gpios`, `rts-gpios`
- `xxx-names`
  - Sometimes naming items is relevant
  - Allows drivers to perform lookups by name rather than ID
  - The order of definition of each item still matters
  - Examples: `gpio-names`, `clock-names`, `reset-names`

```
uart0@4000c000 {
    dmas = <&edma 26 0>, <&edma 27 0>;
    dma-names = "tx", "rx";
    ...
};
```



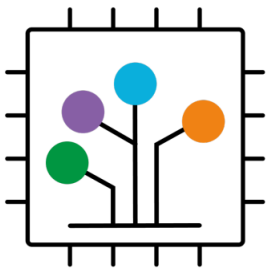
How to validate Device Tree content? 1/2

- `compatible` properties enforce a specific programming model
- OS expect a specific set of properties in each node
  - The syntax is fixed
  - The content is defined (number of items, their size, their meaning)
  - Some properties are mandatory
- How do I check the validity of a DT snippet?
  - How do I avoid losing half a day on a typo?
  - Looking at drivers to understand the DT structure tends to make it OS-specific



How to validate Device Tree content? 2/2

- **Device Tree Specifications** → base Device Tree syntax + number of standard properties.
  - https://www.devicetree.org/specifications/
  - Not sufficient to describe the wide variety of hardware.
- **Device Tree Bindings** → describes how a piece of HW should be described
  - Common bindings are defined in an external repository https://github.com/devicetree-org/dt-schema/tree/main/dtschema/schemas
    - Generic properties: `reg` or
    - Consumer bindings: `interrupts`, `clocks`, `dmas`, etc
  - Device-specific descriptions are in the Linux kernel sources Documentation/devicetree/bindings/



Devicetree Specification

Release v0.3

devicetree.org

13 February 2020



Device Tree bindings

- Bindings are improved as part of the Linux kernel contribution process
- They are carefully reviewed by DT binding maintainers and can only be merged once approved by them
- Need for automated verifications:
  - Legacy: human readable .txt documents, hardly parsable by tools
  - Current norm: YAML-written specifications, easy to parse by humans and tools at the same time!



Device Tree binding: legacy style

Documentation/devicetree/bindings/i2c/i2c-omap.txt

```
I2C for OMAP platforms

-Required properties :
- compatible : Must be
       "ti,omap2420-i2c" for OMAP2420 SoCs
       "ti,omap2430-i2c" for OMAP2430 SoCs
       "ti,omap3-i2c" for OMAP3 SoCs
       "ti,omap4-i2c" for OMAP4+ SoCs
       "ti,am654-i2c", "ti,omap4-i2c" for AM654 SoCs
       "ti,j721e-i2c", "ti,omap4-i2c" for J721E SoCs
       "ti,am64-i2c", "ti,omap4-i2c" for AM64 SoCs
- ti,hwmods : Must be "i2c<n>", n being the instance number (1-
based)
- #address-cells = <1>;
- #size-cells = <0>;

Recommended properties :
- clock-frequency : Desired I2C bus clock frequency in Hz.
Otherwise
  the default 100 kHz frequency will be used.

Optional properties:
- Child nodes conforming to i2c bus binding
```

```
Note: Current implementation will fetch base
address, irq and dma from omap hwmod data base
during device registration.
Future plan is to migrate hwmod data base contents
into device tree blob so that, all the required
data will be used from device tree dts file.

Examples :

i2c1: i2c@0 {
    compatible = "ti,omap3-i2c";
    #address-cells = <1>;
    #size-cells = <0>;
    ti,hwmods = "i2c1";
    clock-frequency = <400000>;
};
```



Device Tree binding: YAML style

Documentation/devicetree/bindings/i2c/ti,omap4-i2c.yaml

```yaml
# SPDX-License-Identifier: (GPL-2.0-only OR BSD-2-
Clause)
%YAML 1.2
---
$id: http://devicetree.org/schemas/i2c/ti,omap4-i2
c.yaml#
$schema: http://devicetree.org/meta-schemas/core.
yaml#

title: I2C controllers on TI's OMAP and K3 SoCs

maintainers:
  - Vignesh Raghavendra <vigneshr@ti.com>

properties:
  compatible:
    oneOf:
      - enum:
          - ti,omap2420-i2c
          - ti,omap2430-i2c
          - ti,omap3-i2c
          - ti,omap4-i2c
      - items:
          - enum:
              - ti,am4372-i2c
              - ti,am64-i2c
              - ti,am654-i2c
              - ti,j721e-i2c
          - const: ti,omap4-i2c

  reg:
    maxItems: 1

  interrupts:
    maxItems: 1

  clocks:
    maxItems: 1

  clock-names:
    const: fck

  clock-frequency: true

  power-domains: true

  "#address-cells":
    const: 1

  "#size-cells":
    const: 0

  ti,hwmods:
    description:
      Must be "i2c<n>", n being [...]
    $ref: /schemas/types.yaml#/definitions/string
    deprecated: true

required:
  - compatible
  - reg
  - interrupts

additionalProperties: false

if:
  properties:
    compatible:
      enum:
        - ti,omap2420-i2c
        - ti,omap2430-i2c
        - ti,omap3-i2c
        - ti,omap4-i2c then:
  properties:
    ti,hwmods:
      items:
        - pattern: "^i2c([1-9])$"
else:
  properties:
    ti,hwmods: false

examples:
  - |
    #include <dt-bindings/interrupt-controller/
irq.h>
    #include <dt-bindings/interrupt-controller/arm-
gic.h>

    main_i2c0: i2c@2000000 {
        compatible = "ti,j721e-i2c", "ti,omap4-
i2c";
        reg = <0x2000000 0x100>;
        interrupts = <GIC_SPI 200
IRQ_TYPE_LEVEL_HIGH>;
    };
```



Validating Device Trees

- `dtc` only does syntactic validation
- YAML bindings allow to do semantic validation
- Linux kernel `make` rules:
  - `make dt_binding_check` verify that YAML bindings are valid, particularly useful if you write examples!
  - `make dtbs_check` validate DTs currently enabled against YAML bindings
- The combination of DTS and bindings growing, it may sometimes be relevant to only check against a subset of matching schema by adding the `DT_SCHEMA_FILES` specifier on the `make` command line:
  - eg. `make DT_SCHEMA_FILES=Documentation/devicetree/bindings/trivial-devices.yaml dtbs_check`
  - Can be used with both `dt_binding_check` and `dtbs_check`



Bindings syntax: base structure

```yaml
# SPDX-License-Identifier: (GPL-2.0-only OR BSD-2-Clause)
%YAML 1.2
---
$id: http://devicetree.org/schemas/<path>/<file-name.yaml>#
$schema: http://devicetree.org/meta-schemas/core.yaml#

title: <Type and name of the device>

maintainers:
  - John Doe <john@doe.com>

description: |
  Some multiline text.

  At an additional indentation level.

# This line is a comment
properties:
  prop-a:
    ...

  prop-b:
    ...
```

Each YAML file defines one DT hierarchical level (up to two when there are children nodes expected)

- `%YAML` defines the expected language version
- `$id` maybe not a real URL, but a unique identifier
- `$schema` refers to the base meta-schema this file should be validated against (in the Github repository mentioned previously)
- `properties:` where the definitions start
- All possible properties should be listed
  - dash-separated lowercase names
  - names followed by a colon ‘`:`’ and a new line
- Every indentation level is 2 spaces
- An empty line between property definitions



Bindings syntax: types

```
properties:
  # A boolean property, basically a yes or no
  pin-x-not-wired:      # pin-x-not-wired;
    type: boolean

  # Expects a single 32-bit numerical value
  start-offset:         # start-offset: <0x1000>;
    $ref: /schemas/types.yaml#/definitions/uint32

  # The suffix already enforces a numerical value!
  # In this case if there is no additional constraint
  # we set the property to 'true'
  my-freq-hz: true      # my-freq-hz = <100000>;

  # Expects an array of 32-bit numerical values
  supported-rates:      # supported-rates = <25>, <50>;
    $ref: /schemas/types.yaml#/definitions/uint32-array

  # A string value is expected
  instruction-set:      # instruction-set = "extended";
    $ref: /schemas/types.yaml#/definitions/string

  # Phandles will be expected
  sampling-lines:       # sampling-lines = <&pioA 1>, <&pioA
5>;
    $ref: /schemas/types.yaml#/definitions/phandle-array

  # Here as well, but no need to repeat the constraint
  # because '-gpios' is a generic suffix
  reset-gpios: true     # reset-gpios = <&gpio SOC_SPEC_IDX>;
```

- Properties must be typed, either with the `type:` or the `ref:` keyword.
  - Boolean properties require no value
  - Numerical values can be signed or unsigned but should always be 32-bit wide
  - Strings should always be fully defined (see next slides)
  - Arrays and matrices are possible as well
- Generic bindings already set the type for many properties:
  - Their values/items numbers can be constrained further
  - The types don't need to be repeated however
- `dt-schema` will enforce a type based on the property name suffix, eg: `-hz`, `-ohms`, `-us`



Bindings syntax: child nodes

```yaml
properties:
  # The sub-node can only be named: child-node
  child-node:
    type: object

patternProperties:
  # The sub-node name is flexible, eg: child@1000, child@2a, etc
  "^child@[a-f0-9]+$":
    type: object
```

- From a yaml-schema perspective, children nodes are just another property
- A specific type shall however be enforced:
  - `type: object`
- Under the main `properties` keyword, property/sub-node names are fixed
  - If the sub-node name is dynamic, we shall define it under another top-level keyword, `patternProperties` and use pattern-matching regexes for the naming



Bindings syntax: expressing constraints

Besides defining precisely the different properties and their type, the content of the property values must also be constrained.

- All properties can get an additional `description` parameter, which is only readable by humans
- We try to maximize the constraints to minimize human errors
- One new line per constraint



Bindings syntax: numerical constraints

```yaml
properties:
  # The numerical value is bounded
  # This is valid:
  # frequency-hz = <100000>;
  # frequency-hz = <0x40000>; /* 262144 Hz */
  # This is not:
  # frequency-hz = <0>;
  # frequency-hz = <&gpio 10>;
  frequency-hz:
    minimum: 10000
    maximum: 400000
    default: 100000

  # This is an array with either 1 or 2 members
  # This is valid:
  # cs-gpios = <&gpioA 1>;
  # cs-gpios = <&gpioA 1>, <gpioA 5>;
  # This is not:
  # cs-gpios = <&gpioA 1>, <gpioA 5>, <gpioA 6>;
  # cs-gpios = <50>;
  cs-gpios:
    minItems: 1
    maxItems: 2
```

- Example of constraints:
  - `minimum:`/`maximum:` min/max values for a single value
  - `default:` for a default value
  - `minItems:`/`maxItems:` min/max number of items in an array



Bindings syntax: lists and dictionaries

```
properties:
  # This is a very common compatible definition
  # The only allowed combinations are (order matters):
  # compatible = "vendor1,compat", "generic,compat";
  # compatible = "vendor2,compat", "generic,compat";
  # compatible = "legacy-compat";
  compatible:
    oneOf:
      - items:
          - enum:
              - vendor1,compat
              - vendor2,compat
          - const: generic,compat
      - items:
          - const: legacy-compat

  # Property name is known by dt-schema, type will be
inferred
  # No need for minItems/maxItems, 2 will be implied from
  # the main items list!
  clocks:
    items:
      - description: Interconnect
      - description: External bus

  # This is valid:   strength = <0>, <5>;
  # This is invalid: strength = <0>;
  #                  strength = <0>, <8>;
  strength:
    $ref: /schemas/types.yaml#/definitions/uint32-array
    minItems: 2
    maxItems: 2
    items:
      maximum: 5
```

- Expressing several possible property values (works with numbers and strings):
  - Force a single expected value: `const`
  - Allow taking one value from a list: `enum`
    - watch out the indentation: 2 spaces from the previous keyword and a dash
- `const`/`enum` can be grouped within an `items` list, where each `items` sub-entry must be observed
- We can build abstract conditional lists (eg. on top of `items` rather than proper values like with `const`/`enum`:
  - XOR using `oneOf`
  - OR using `anyOf`
  - AND using `allOf`



Bindings syntax: referencing other bindings

```
# All properties/constraints defined in generic-
controller.yaml
# will apply (but they can be tuned/overwritten below)
allOf:
  - $ref: generic-controller.yaml

properties:
  # Tune a property defined in generic-controller.yaml
  prop-a:
    maximum: 1

  # Allow a new, more specific property
  vendor,specific-prop: true

  # common-child-constraints.yaml will enforce a base set
of
  # properties and rules
  child-node:
    type: object
    $ref: common-child-constraints.yaml
```

- It is possible to write “common” constraints in a YAML file and refer to it
  - Very usual when describing a certain type of controller
    - Refer to the generic constraints with a top-level `allOf`
    - Add constraints which are specific to the hardware implementation
  - Possible to constrain children nodes by referencing another YAML file



Bindings syntax: altering on presence of properties

```yaml
properties:
  compatible:
    enum:
      - compat1
      - compat2

  prop-a: true

  prop-b: true

  prop-c: true

dependencies:
  prop-a: [ 'prop-b' ]
  prop-b: [ 'prop-a' ]

allOf:
  - if:
      properties:
        compatible:
          contains:
            const: compat1
    then:
      properties:
        prop-c: false
```

- Sometimes more dynamic descriptions are needed
  - Dependencies between properties
    - A property may be needed if there is another property
    - If both or none shall be present, the dependency should be expressed twice (in both directions)
  - Changing constraints based on a property
    - Can be expressed using `if`/`else` statements under the top-level `allOf`
    - Typical case: a `compatible` implies tweaking a constraint



Bindings syntax: enforcing correct properties only

```yaml
allOf:
  - $ref: generic-file.yaml

properties:
  prop-a: true

  prop-b: true

  child-node:
    type: object
    properties:
      prop-c: true
      prop-d: true

    required:
      - prop-c

    # No additional property than the ones above
    # will be allowed inside child-node
    additionalProperties: false

required:
  - prop-a

# Only properties defined below or coming from
# generic-file.yaml will be allowed unevaluatedProperties:
false
```

- YAML files list properties and add constraints to them
  - It is still possible to add undefined properties
  - It is still possible to forget defining a mandatory property
- We need further constraints to spot typos and unexpected properties
  - `required` forces the presence
  - `additionalProperties` prevents any property not defined in **this** file to be used
  - `unevaluatedProperties` prevents any property not defined in this file nor referenced (through `allOf` or) to be used



Bindings syntax: validating your own bindings

```yaml
properties:
  prop-a: true
  prop-b: true

  child-node:
    type: object
    additionalProperties: false

required:
  - prop-a

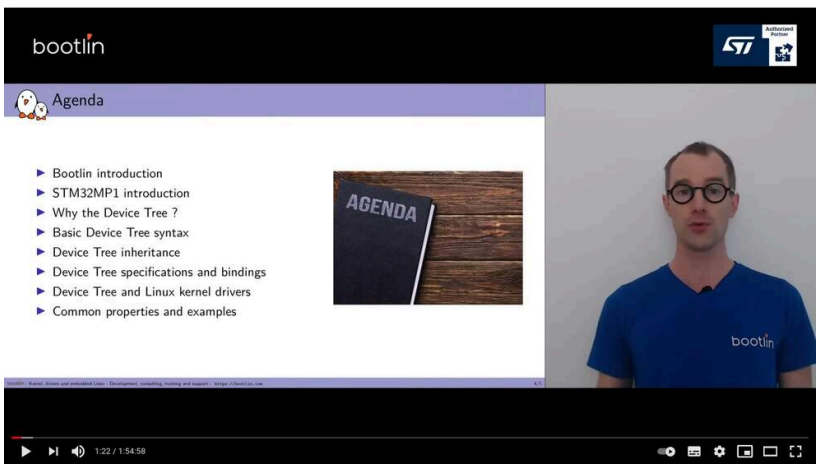
unevaluatedProperties: false

example:
  - |
    node@1000 {
      prop-a;
    };
```

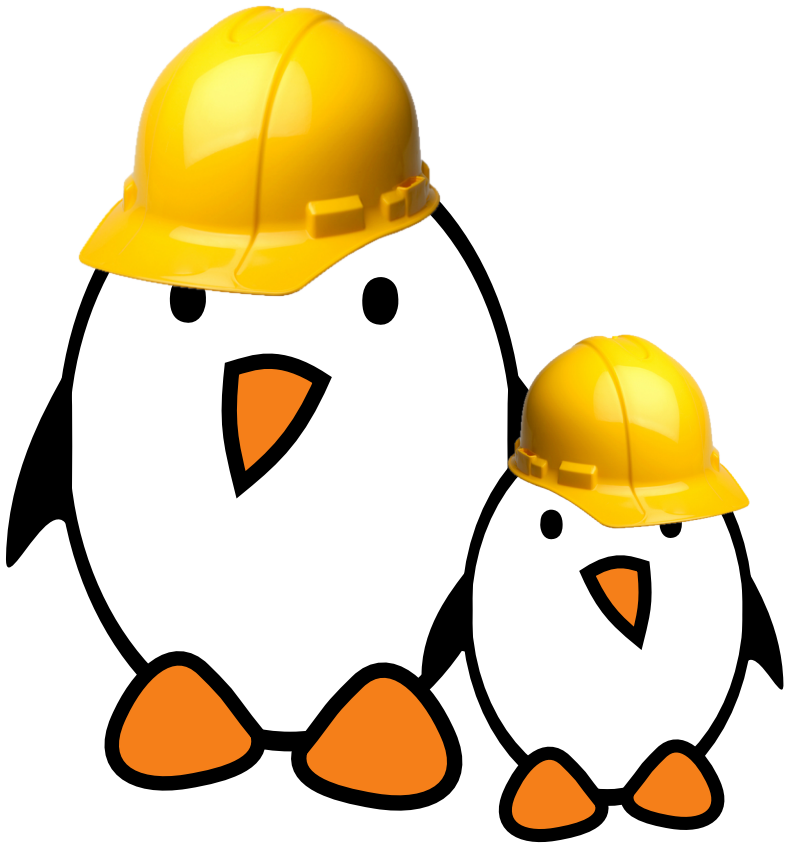
- It is very recommended to test your bindings before testing your DTS
  - Add `examples` at the end of your file!
  - Examples are indented with 4 spaces

References

- Device Tree 101 webinar, Thomas Petazzoni (2021):
  Slides: https://bootlin.com/blog/device-tree-101-webinar-slides-and-videos/
  Video: https://youtu.be/a9CZ1Uk3OYQ
- Kernel documentation
  - driver-api/driver-model/
  - devicetree/
  - filesystems/sysfs
- https://devicetree.org
- The kernel source code
  - Full of examples of other drivers!
  - Reference DT binding implementation: Documentation/devicetree/bindings/example-schema.yaml



Practical lab - Describing hardware devices



- Browse and update Device Trees.
- Use GPIO LEDs.
- Modify the Device Tree to enable an I2C controller and describe an I2C device.
- Write a yaml binding to validate a device description.